

# Ada2MS: A Hybrid Optimization Algorithm Based on Exponential Mixing of Elementwise and Global Second-Moment Estimates

Meng Zhu<sup>a</sup>, Quan Xiao<sup>a</sup>, Weidong Min<sup>b,c,d,\*</sup>

<sup>a</sup>*School of Information Management and Mathematics, Jiangxi University of Finance and Economics, Nanchang 330032, China*

<sup>b</sup>*School of Mathematics and Computer Science, Nanchang University, Nanchang 330031, China*

<sup>c</sup>*Institute of Metaverse, Nanchang University, Nanchang 330031, China*

<sup>d</sup>*Jiangxi Provincial Key Laboratory of Virtual Reality, Nanchang 330031, China*

---

## Abstract

Optimization algorithms are core methods by which machine learning models iteratively minimize loss functions, update parameters, learn from data, and improve performance. Momentum SGD and AdamW represent two important optimization paradigms. AdamW produces stable updates and usually has strong robustness across training scenarios, but its generalization performance is sometimes weaker than that of momentum methods. Momentum SGD can often obtain better generalization after careful tuning, but it is more sensitive to gradient-scale variation and hyperparameter settings. To balance the strengths and weaknesses of the two paradigms, this paper proposes Ada2MS, an optimization algorithm that achieves a smooth transition between AdamW-like behavior and momentum-SGD-like behavior through continuous exponential interpolation between elementwise second-moment estimates and global second-moment estimates. On the visual tasks evaluated in this study, Ada2MS obtains competitive results under a unified optimizer-comparison protocol. The code will be released at <https://github.com/mengzhu0308/Ada2MS>.

**Keywords:** Optimization algorithm, Momentum SGD, AdamW, Global second-moment estimate, Exponential mixing

---



---

\*Corresponding author

Email addresses: zhumeng@jxufe.edu.cn (Meng Zhu), xiaoquan@foxmail.com (Quan Xiao), minweidong@ncu.edu.cn (Weidong Min)

## 1. Introduction

Optimization algorithms are central to improving model convergence efficiency. Their significance can be summarized from three perspectives. First, they reduce hardware cost. Improved optimization algorithms can reduce the number of convergence steps and improve training efficiency under a fixed hardware budget. Second, they promote scientific discovery. Differentiable optimization frameworks play key roles in protein structure prediction [1], materials design [2], and controllable nuclear fusion simulation [3]. Third, they reduce energy consumption and carbon emissions. Through algorithmic and system-engineering optimization, the pretraining of LLaMA 2-70B achieved total carbon emissions of 539 tCO<sub>2</sub>e [4].

AdamW [5] remains a widely used optimization algorithm in current deep learning. This is mainly because it can automatically handle differences in gradient scale and usually provides stable parameter updates across training scenarios. In particular, the root-mean-square norm of the AdamW update is bounded, and decoupled weight decay also helps control parameter magnitude. AdamW therefore has strong practical stability.

However, adaptive update steps may also have adverse effects. From the perspective of noise, adaptive steps introduce anisotropic noise, suppressing exploration in sharp regions and amplifying perturbations in flat regions, which makes the optimizer more likely to prefer sharp extrema [6, 7]. By contrast, stochastic gradient descent (SGD) with momentum [8], when it can train stably, is often more likely to obtain flat solutions with better generalization. However, it cannot automatically handle gradient-scale variation and therefore often requires more careful hyperparameter tuning for recent architectures [9, 10].

Based on this trade-off, this paper proposes Ada2MS. Unlike discrete switching, Ada2MS performs smooth interpolation between AdamW-like behavior and momentum-SGD-like behavior through exponential interpolation between elementwise second-moment estimates and global second-moment estimates, as detailed in Algorithm 1. The design aims to gradually introduce more SGD-oriented update characteristics in the later stage of training while maintaining update-scale stability as much as possible.

The remainder of this paper is organized as follows. Section 2 reviews existing research on optimization algorithms in deep learning. Section 3 presents the proposed Ada2MS algorithm in detail. Section 4 presents the experimental protocol, results, and analysis. The final section summarizes the paper and discusses future work.

## 2. Related work

In 1986, Rumelhart *et al.* [11] first presented the backpropagation algorithm, which can compute exact gradients for multilayer networks over a full training set in a single pass. Although this algorithm is theoretically complete, the computation and storage cost of one iteration becomes restrictive for large-scale datasets. To overcome this limitation, Rumelhart *et al.* [11] also proposed SGD, which replaces full gradients with mini-batch gradient approximations. It randomly selects a mini-batch at each iteration to estimate the gradient and thus substantially reduces the cost of one parameter update. Nevertheless, the practical convergence behavior of SGD still has three main weaknesses. (1) Absence of momentum. The update direction depends completely on the current gradient, which causes persistent oscillations on narrow or high-curvature error surfaces and slows convergence. (2) Imbalanced step sizes. When the gradient scales of different parameters differ significantly, a unified learning rate cannot simultaneously accommodate sparse and dense features, causing some parameters to stop updating and others to fluctuate strongly. (3) Poor learning-rate robustness. A learning rate that is too large can make the objective diverge, whereas a learning rate that is too small can stall updates.

To address the first and third weaknesses of SGD, Sutskever *et al.* [8] proposed an exponentially weighted momentum mechanism for SGD. This mechanism integrates historical gradient information into the current update direction, suppressing oscillations, accelerating movement across flat regions, and improving robustness to the learning rate. Gitman *et al.* [12] used stochastic differential equations to model how different momentum values affect escape from saddle points and convergence speed, providing practical ranges for momentum. Ramezani-Kebrya *et al.* [13] derived stability and generalization-error bounds for SGD with momentum, theoretically showing

that momentum not only accelerates training but can also reduce overfitting risk. Zhao *et al.* [14] proposed a “normalization plus momentum” combination and proved that it can maintain an  $O(1/T)$  convergence rate under anisotropic noise, thereby effectively reducing directional oscillation in SGD.

To address the second and third weaknesses of SGD, Duchi *et al.* [15] proposed AdaGrad, which uses the cumulative sum of historical squared gradients to realize per-parameter adaptive learning steps and significantly improves convergence efficiency in sparse-data scenarios. However, because AdaGrad uses the cumulative historical squared gradient as the denominator, the learning step decreases monotonically during training and approaches zero, causing late-stage updates to almost stop. To alleviate this bottleneck, Tieleman *et al.* [16] proposed RMSProp, which replaces the global accumulation mechanism with an exponential moving average of squared gradients and avoids vanishing learning steps.

Although momentum SGD and RMSProp respectively improve local weaknesses of SGD, their advantages are not combined. Adam, proposed by Kingma *et al.* [17], integrates the momentum mechanism and adaptive step-size strategy and proposes bias correction to offset the inherent bias of exponentially weighted averages, further improving learning-rate robustness. AMSGrad, proposed by Reddi *et al.* [18], replaces Adam’s sliding average of the second moment with the historical maximum of the gradients to avoid convergence to local extrema with poor generalization. AdamW, proposed by Loshchilov *et al.* [5], decouples weight decay from the parameter update of the optimizer, making it a regularization term independent of the target update and thereby improving generalization and making hyperparameter tuning more robust. Adafactor, proposed by Shazeer *et al.* [19], uses low-rank factorization of the second moment to achieve sublinear memory usage, effectively reducing memory requirements for training large models. Sophia, proposed by Liu *et al.* [20], presents lightweight diagonal Hessian estimates for large language model pretraining, achieving faster convergence and lower computational cost than Adam. Sophia uses a diagonal Hessian approximation instead of the full Hessian matrix, reducing computation but sacrificing curvature-correlation information among parameters. The Hessian is estimated every  $k$  steps, which limits average computational overhead to within 5% of

gradient computation, but it may still cause delayed dynamic responses, and the estimation variance may be amplified in early training or small-batch scenarios, affecting stability.

Although Adam and its variants can converge faster to extrema, they are less likely than SGD to find flat extrema with favorable generalization. To address this limitation, Keskar *et al.* [6] proposed **Switching from Adam to SGD (SWATS)**, a hybrid optimization algorithm whose key innovation is to adaptively switch from Adam to SGD during training, eliminating manual adjustment of the switching time and SGD learning rate. To address the abnormal variance of Adam’s adaptive learning steps in the initial training stage caused by insufficient samples, Rectified Adam (RAdam), proposed by Liu *et al.* [21], mathematically estimates the confidence of the gradient degrees of freedom  $\rho$ . If  $\rho > 4$ , it enables a variance-rectified adaptive learning rate; if  $\rho \leq 4$ , it falls back to a non-adaptive momentum mechanism. AdaX, proposed by Li *et al.* [22], changes Adam’s exponential decay mechanism for the second moment into exponential growth, giving historical gradient information exponential long-term memory and thereby alleviating Adam’s tendency in the stationary training phase to produce excessively large steps due to sensitivity to small gradients. Xie *et al.* [7] used a dynamical framework to show that Adam escapes saddle points faster but is less likely than SGD to find flat minima with favorable generalization, and they proposed the **Adaptive Inertia (AdaI)** optimizer. Zhu *et al.* [23] proposed AdamNX, whose core innovation is a new exponential decay rate for second-moment estimation, so that the strength of learning-step correction gradually weakens as training proceeds. The optimizer therefore degenerates into stochastic gradient descent with momentum in the stationary phase, improving late-stage training stability and potentially improving generalization.

In addition, both Adam and AdamW require first- and second-moment estimates to be stored for each parameter, leading to relatively high memory and computational cost. To overcome this limitation, Chen *et al.* [24] proposed Lion, which uses a sign-momentum mechanism to effectively reduce memory requirements and computation while matching or exceeding Adam on many tasks. However, when the batch size is below 64, Lion performs worse than AdamW. More recently, Jordan *et al.* [25] proposed Muon specifically for matrix parameters. Muon uses Newton–Schulz iteration

to approximately orthogonalize the momentum update matrix, significantly improving training efficiency. Nevertheless, Muon still has several limitations: (1) its computation time is higher than Adam’s; (2) when matrix shards are distributed across multiple devices, gradients from all shards must first be aggregated by all-reduce before the update can be computed, making independent parallel updates on each device difficult and introducing additional communication overhead; (3) when a model is extremely over-trained, Muon behaves similarly to Adam; and (4) Muon must be used in combination with Adam.

In summary, the development of deep learning optimizers has consistently focused on improving convergence efficiency, reducing computational cost, and enhancing hyperparameter robustness. Therefore, mixing Adam-like and SGD-like behavior is not a problem setting first introduced by this paper. The contribution boundary of this paper should be defined more precisely: Ada2MS does not claim to be the first to combine Adam-like and SGD-like optimization behavior. Instead, it realizes a smooth transition through continuous interpolation between elementwise second-moment normalization and global second-moment normalization. This differs from discrete switching methods such as SWATS and also from methods such as AdaBound that gradually approach SGD behavior through effective learning-rate bounds.

### 3. Proposed optimization algorithm

#### 3.1. Derivation assumptions

Let

$$\mathbf{g}_t = \nabla f(\boldsymbol{\theta}_{t-1}) + \boldsymbol{\xi}_t \quad (1)$$

where  $\nabla f(\boldsymbol{\theta}_{t-1})$  denotes the full gradient, namely the gradient of  $\boldsymbol{\theta}_{t-1}$  computed using all samples  $\mathcal{D}$ .  $\mathbf{g}_t$  denotes the stochastic gradient, namely the gradient of  $\boldsymbol{\theta}_{t-1}$  computed by randomly drawing a subset  $\mathcal{R} \subset \mathcal{D}$ .  $\boldsymbol{\xi}_t$  denotes the noise introduced by random sampling. Therefore,  $\mathbf{g}_t$  is only an approximate estimate of  $\nabla f(\boldsymbol{\theta}_{t-1})$ . In the experiments in Section 4, the mini-batch gradient serves as this stochastic estimator. The purpose of this section is mainly to provide a mechanism-level stability analysis under simplified assumptions, rather than a complete convergence theorem for mini-batch nonconvex

Ada2MS. Because sampling is random,  $\xi_t$  is isotropic, and the noises  $\xi_t$  and  $\xi_{t+i}$  at different times are independent. For the semi-qualitative and semi-quantitative analysis below, we make the following assumptions:

**Hypothesis 1.** *The components of the full gradient are independent and identically distributed, and the components of the noise vector are independent and identically distributed.*

**Hypothesis 2.**  $\xi_t \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{1})$ .

**Hypothesis 3.** *The full gradient is a random variable and is stationary.*

**Hypothesis 4.** *The full gradient  $\nabla f(\theta_{t-1})$  and the random noise  $\xi_t$  are independent.*

Based on Assumptions 1 and 2, Equation (1) can be written as

$$\mathbf{g}_t = \nabla f(\theta_{t-1}) + \sigma \xi \quad (2)$$

where  $\xi$  denotes the standard normal distribution. Based on Assumptions 1 and 3, we have

$$\begin{cases} \mathbb{E}_t[\nabla f(\theta_{t-1})] = \mu_{\nabla f} \mathbf{1}, \\ \text{Var}_t[\nabla f(\theta_{t-1})] = \sigma_{\nabla f}^2 \mathbf{1} \end{cases} \quad (3)$$

Based on Assumptions 1 and 4, we have

$$\text{Cov}[\nabla f(\theta_{t-1}), \xi_t] = \mathbf{0} \quad (4)$$

### 3.2. Statistical dimensionality of momentum SGD

For momentum SGD $\mathbf{W}$ , the update rule is

$$\begin{cases} \mathbf{m}_t = \beta \mathbf{m}_{t-1} + \mathbf{g}_t, \\ \theta_t = \theta_{t-1} - \eta_t (\mathbf{m}_t + \lambda \theta_{t-1}) \end{cases} \quad (5)$$

where  $\beta$  denotes the momentum coefficient,  $\eta_t$  denotes the learning rate, and  $\lambda$  denotes the weight-decay rate. Iterating  $\mathbf{m}_t$  gives

$$\mathbf{m}_t = \sum_{i=0}^{t-1} \beta^i \mathbf{g}_{t-i} \quad (6)$$

Substituting Equation (2) into Equation (6) gives

$$\mathbf{m}_t = \sum_{i=0}^{t-1} \beta^i (\nabla f(\boldsymbol{\theta}_{t-1-i}) + \sigma \boldsymbol{\xi}) \quad (7)$$

The expectation of the momentum term is

$$\begin{aligned} \mathbb{E}[\mathbf{m}_t] &= \mathbb{E} \left[ \sum_{i=0}^{t-1} \beta^i (\nabla f(\boldsymbol{\theta}_{t-1-i}) + \sigma \boldsymbol{\xi}) \right] = \sum_{i=0}^{t-1} \beta^i \mathbb{E}[\nabla f(\boldsymbol{\theta}_{t-1-i}) + \sigma \boldsymbol{\xi}] \\ &= \sum_{i=0}^{t-1} \beta^i \mathbb{E}[\nabla f(\boldsymbol{\theta}_{t-1-i})] = \frac{1 - \beta^t}{1 - \beta} \mu_{\nabla f} \mathbf{1} \end{aligned} \quad (8)$$

The variance of the momentum term is

$$\begin{aligned} \text{Var}[\mathbf{m}_t] &= \sum_{i=0}^{t-1} \beta^{2i} \text{Var}[\nabla f(\boldsymbol{\theta}_{t-1-i}) + \sigma \boldsymbol{\xi}] \\ &= \sum_{i=0}^{t-1} \beta^{2i} (\text{Var}[\nabla f(\boldsymbol{\theta}_{t-1-i})] + \text{Var}[\sigma \boldsymbol{\xi}] + 2\text{Cov}[\nabla f(\boldsymbol{\theta}_{t-1-i}), \sigma \boldsymbol{\xi}]) \\ &= \sum_{i=0}^{t-1} \beta^{2i} (\sigma_{\nabla f}^2 + \sigma^2) \mathbf{1} = \frac{1 - \beta^{2t}}{1 - \beta^2} (\sigma_{\nabla f}^2 + \sigma^2) \mathbf{1} \end{aligned} \quad (9)$$

The root-mean-square (RMS) norm of the momentum term is

$$\begin{aligned} \|\mathbf{m}_t\|_{\text{RMS}} &= \sqrt{\frac{1}{d} \mathbb{E}[\|\mathbf{m}_t\|^2]} = \sqrt{\frac{1}{d} \mathbb{E} \left[ \sum_{i=1}^d m_{t,i}^2 \right]} = \sqrt{\frac{1}{d} \sum_{i=1}^d \mathbb{E}[m_{t,i}^2]} \\ &= \sqrt{\mathbb{E}[m_{t,i}^2]} = \sqrt{\left( \frac{1 - \beta^t}{1 - \beta} \mu_{\nabla f} \right)^2 + \frac{1 - \beta^{2t}}{1 - \beta^2} (\sigma_{\nabla f}^2 + \sigma^2)} \end{aligned} \quad (10)$$

Here  $d$  denotes the vector dimension.

### 3.3. Statistical dimensionality of AdamW

For AdamW, the update rule is

$$\begin{cases} \mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \\ \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \\ \widehat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \widehat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \\ \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta_t \left( \frac{\widehat{\mathbf{m}}_t}{\epsilon + \sqrt{\widehat{\mathbf{v}}_t}} + \lambda \boldsymbol{\theta}_{t-1} \right) \end{cases} \quad (11)$$

where  $\beta_1$  and  $\beta_2$  denote the exponential decay rates for the first- and second-moment estimates, respectively, and  $\epsilon$  is a small constant used to prevent division by zero. Iterating  $\mathbf{m}_t$  gives

$$\mathbf{m}_t = (1 - \beta_1) \sum_{i=0}^{t-1} \beta_1^i \mathbf{g}_{t-i} = (1 - \beta_1) \sum_{i=0}^{t-1} \beta_1^i (\nabla f(\boldsymbol{\theta}_{t-1-i}) + \sigma \boldsymbol{\xi}) \quad (12)$$

The expectation of the first-moment estimate in AdamW is

$$\mathbb{E}[\mathbf{m}_t] = (1 - \beta_1) \sum_{i=0}^{t-1} \beta_1^i \mathbb{E}[\nabla f(\boldsymbol{\theta}_{t-1-i})] = (1 - \beta_1^t) \mu_{\nabla f} \mathbf{1} \quad (13)$$

The variance of the first-moment estimate in AdamW is

$$\begin{aligned} \text{Var}[\mathbf{m}_t] &= (1 - \beta_1)^2 \sum_{i=0}^{t-1} \beta_1^{2i} (\text{Var}[\nabla f(\boldsymbol{\theta}_{t-1-i})] + \sigma^2 \mathbf{1}) \\ &= \frac{1 - \beta_1}{1 + \beta_1} (1 - \beta_1^{2t}) (\sigma_{\nabla f}^2 + \sigma^2) \mathbf{1} \end{aligned} \quad (14)$$

Iterating  $\mathbf{v}_t$  gives

$$\begin{aligned} \mathbf{v}_t &= (1 - \beta_2) \sum_{i=0}^{t-1} \beta_2^i \mathbf{g}_{t-i}^2 = (1 - \beta_2) \sum_{i=0}^{t-1} \beta_2^i (\nabla f(\boldsymbol{\theta}_{t-1-i}) + \sigma \boldsymbol{\xi})^2 \\ &= (1 - \beta_2) \sum_{i=0}^{t-1} \beta_2^i ((\nabla f(\boldsymbol{\theta}_{t-1-i}))^2 + 2 \nabla f(\boldsymbol{\theta}_{t-1-i}) \odot \sigma \boldsymbol{\xi} + \sigma^2 \boldsymbol{\xi}^2) \end{aligned} \quad (15)$$

The expectation of the second-moment estimate in AdamW is

$$\mathbb{E}[\mathbf{v}_t] = (1 - \beta_2^t) (\mu_{\nabla f}^2 + \sigma_{\nabla f}^2 + \sigma^2) \mathbf{1} \quad (16)$$

The RMS norm of the AdamW update vector, excluding the update vector introduced by weight decay, is

$$\begin{aligned}
\left\| \frac{\widehat{\mathbf{m}}_t}{\epsilon + \sqrt{\widehat{\mathbf{v}}_t}} \right\|_{\text{RMS}} &= \sqrt{\frac{1}{d} \mathbb{E} \left[ \left\| \frac{\widehat{\mathbf{m}}_t}{\epsilon + \sqrt{\widehat{\mathbf{v}}_t}} \right\|^2 \right]} = \sqrt{\frac{1}{d} \sum_{i=1}^d \mathbb{E} \left[ \left( \frac{\widehat{m}_{t,i}}{\epsilon + \sqrt{\widehat{v}_{t,i}}} \right)^2 \right]} \\
&\approx \sqrt{\frac{1}{d} \sum_{i=1}^d \frac{\mathbb{E}[\widehat{m}_{t,i}^2]}{\left( \epsilon + \sqrt{\mathbb{E}[\widehat{v}_{t,i}]} \right)^2}} \\
&= \frac{\sqrt{\frac{(1-\beta_1)(1+\beta_1')}{(1+\beta_1)(1-\beta_1')} (\sigma_{\nabla f}^2 + \sigma^2) + \mu_{\nabla f}^2}}{\epsilon + \sqrt{\sigma_{\nabla f}^2 + \sigma^2 + \mu_{\nabla f}^2}} \\
&\approx \frac{\sqrt{\frac{(1-\beta_1)(1+\beta_1')}{(1+\beta_1)(1-\beta_1')} (\sigma_{\nabla f}^2 + \sigma^2) + \mu_{\nabla f}^2}}{\sqrt{\sigma_{\nabla f}^2 + \sigma^2 + \mu_{\nabla f}^2}}
\end{aligned} \tag{17}$$

### 3.4. Comparing the update stability of momentum SGD and Adam from the RMS norm

Define the generalized signal-to-noise ratio (SNR) as

$$\text{SNR} = \frac{\mu_{\nabla f}^2}{\sigma_{\nabla f}^2 + \sigma^2} \tag{18}$$

Equation (18) measures the strength of the full-gradient mean relative to the total fluctuation of the stochastic gradient, namely the proportion associated with the true signal direction. Substituting the SNR into Equation (10) gives

$$\|\mathbf{m}_t\|_{\text{RMS}} = \sigma_g \sqrt{\left( \frac{1-\beta^t}{1-\beta} \right)^2 \text{SNR} + \frac{1-\beta^{2t}}{1-\beta^2}} \tag{19}$$

It is easy to obtain  $\lim_{\text{SNR} \rightarrow \infty} = \infty$ . Therefore, the RMS norm of the momentum term has no finite upper bound, and momentum SGD may be unstable during training.

Substituting the SNR into Equation (17) gives

$$\left\| \frac{\widehat{\mathbf{m}}_t}{\epsilon + \sqrt{\widehat{\mathbf{v}}_t}} \right\|_{\text{RMS}} = \frac{\sqrt{\frac{(1-\beta_1)(1+\beta_1')}{(1+\beta_1)(1-\beta_1')} + \text{SNR}}}{\sqrt{1 + \text{SNR}}} \tag{20}$$

It is easy to obtain that  $\left\| \frac{\hat{m}_t}{\epsilon + \sqrt{v_t}} \right\|_{\text{RMS}}$  increases monotonically with SNR and has range  $(\sqrt{\frac{1-\beta_1}{1+\beta_1}}, 1)$ . Therefore, the RMS norm of the AdamW update vector has a finite upper bound and is more robust to SNR. AdamW is therefore more stable during training.

In summary, from the perspective of the RMS norm, AdamW updates are more stable and more robust to noise than momentum SGD updates. However, if both optimizers can train stably, AdamW tends to converge to sharp points with slightly worse generalization, whereas momentum SGD tends to converge to flat points with slightly better generalization. The proof is not expanded here; readers may refer to [6, 7]. Therefore, momentum SGD and AdamW have different advantages in different training scenarios.

### 3.5. Proposed Ada2MS optimizer

To address the limited robustness of momentum SGD and the tendency of AdamW to prefer sharp solutions, this paper proposes Ada2MS, which aims to flexibly control the mixing ratio between AdamW-like updates and momentum-SGD-like updates, as shown in Algorithm 1. When  $\alpha_t = 1$ , Ada2MS degenerates into AdamW. When  $\alpha_t = 0$ , Ada2MS degenerates into momentum SGD. When  $\alpha_t \in (0, 1)$ , Ada2MS continuously interpolates between the two.

As  $\alpha_t$  decays from 1 to 0, the momentum-SGD component gradually increases, whereas the RMS norm of the pure momentum-SGD update has no upper bound. To alleviate this issue, this paper proposes a global second-moment estimate:

$$n_{l,t} = \hat{\beta}_{2,t} n_{l,t-1} + (1 - \hat{\beta}_{2,t}) \|\mathbf{g}_{l,t}\|_2^2, \quad \hat{\beta}_{2,t} = \frac{\beta_2 - \beta_2^t}{1 - \beta_2^t} \quad (21)$$

Therefore, the switching term is not written as  $s_{l,t} = (\mathbf{v}_{l,t})^{\alpha_t}$ , but is instead defined as a mixture of the elementwise second moment and the global second moment:

$$s_{l,t} = (\mathbf{v}_{l,t})^{\alpha_t} \left( \frac{n_{l,t}}{\|\mathbf{g}_{l,t}\|} \right)^{1-\alpha_t} \quad (22)$$

Equation (22) also helps align the effective update scale during the transition and reduces oscillations caused by abrupt switching.

We next analyze why, under the above assumptions, introducing the global second-moment estimate helps keep the RMS norm of the update bounded for all  $\alpha_t \in [0, 1]$ .

Consider the extreme case  $\alpha_t = 0$ . If the RMS norm of the update can be controlled in this case, the intermediate cases are usually easier to control. For simplicity, the core update steps at  $\alpha_t = 0$  are first written and rewritten with bias-correction terms:

$$\begin{cases} \mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \\ n_t = \beta_2 n_{t-1} + (1 - \beta_2) \|\mathbf{g}_t\|_2^2, \\ \widehat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \widehat{n}_t = \frac{n_t}{1 - \beta_2^t}, \\ \boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta_t \frac{\widehat{\mathbf{m}}_t}{\epsilon + \sqrt{\frac{\widehat{n}_t}{d}}} \end{cases} \quad (23)$$

Computing  $\mathbb{E}[\|\mathbf{g}_t\|^2]$  gives

$$\begin{aligned} \mathbb{E}[\|\mathbf{g}_t\|^2] &= \mathbb{E}[\|\nabla f(\boldsymbol{\theta}_{t-1})\|^2] + \sigma^2 \mathbb{E}[\|\boldsymbol{\xi}\|^2] + 2\sigma \mathbb{E}[\langle \nabla f(\boldsymbol{\theta}_{t-1}), \boldsymbol{\xi} \rangle] \\ &= d(\sigma_{\nabla f}^2 + \mu_{\nabla f}^2) + d\sigma^2 \end{aligned} \quad (24)$$

The recurrence of the expectation of the global second-moment estimate is

$$\mathbb{E}[n_t] = \beta_2 \mathbb{E}[n_{t-1}] + (1 - \beta_2) \mathbb{E}[\|\mathbf{g}_t\|^2] \quad (25)$$

Iterating Equation (25) gives

$$\mathbb{E}[n_t] = (1 - \beta_2^t)(d(\sigma_{\nabla f}^2 + \mu_{\nabla f}^2) + d\sigma^2) \quad (26)$$

Therefore, the RMS norm of the update is

$$\left\| \frac{\widehat{\mathbf{m}}_t}{\epsilon + \sqrt{\widehat{n}_t/d}} \right\|_{\text{RMS}} \approx \frac{\sqrt{\frac{(1-\beta_1)(1+\beta_1^t)}{(1+\beta_1)(1-\beta_1^t)}(\sigma_{\nabla f}^2 + \sigma^2) + \mu_{\nabla f}^2}}{\sqrt{\sigma_{\nabla f}^2 + \sigma^2 + \mu_{\nabla f}^2}} \quad (27)$$

Equation (27) has the same form as Equation (17). Under the assumptions above, this indicates that the proposed global second-moment estimate helps keep the RMS norm of the Ada2MS update bounded for all  $\alpha_t \in [0, 1]$ . It should be emphasized that this section mainly provides a mechanism-level stability analysis, rather than a complete convergence theorem for mini-batch nonconvex Ada2MS.

Switching-exponent schedule. Compared with AdamW, Ada2MS has an additional hyperparameter  $\alpha_t$ , which is set according to Equation (28):

$$\alpha_t = \begin{cases} 1, & t \leq 0.6T, \\ 1 - \frac{t-0.6T}{0.4T}, & t > 0.6T \end{cases} \quad (28)$$

Equation (28) gives a simple empirical setting rather than a theoretically optimal schedule. The intuition is to maintain strong AdamW-like behavior in the early and middle stages of training for rapid search, and then gradually introduce stronger momentum-SGD-like behavior in the later stage for finer-grained search. A more complete experimental study should also include sensitivity analysis of the switching start point.

---

Algorithm 1: Ada2MS optimizer

---

**Input<sub>1</sub>:** Training sample set  $\mathcal{D}$ .  
**Input<sub>2</sub>:** Parameters to be optimized  $(\theta_1, \theta_2, \dots, \theta_L)$ , first moments  $(m_1, m_2, \dots, m_L)$ , and second moments  $(v_1, v_2, \dots, v_L)$ .  
**Input<sub>3</sub>:** Learning rate  $\eta_t$ , weight-decay rate  $\lambda$ , exponential decay rates  $(\beta_1, \beta_2) = (0.9, 0.99)$ ,  $\epsilon = 1 \times 10^{-12}$ , switching exponent  $\alpha_t$ , and total number of iterations  $T$ .  
**Output:** Extremal point  $(\theta_1^*, \theta_2^*, \dots, \theta_L^*)$ .

```

1 for  $t = 1$  to  $T$  do
2   Randomly draw a training subset:  $\mathcal{R}_t \subseteq \mathcal{D}$ ;
   /* Forward propagation, where  $|\mathcal{R}_t|$  denotes the random batch size */
3   Compute the loss:  $\mathcal{L}_t = \frac{1}{|\mathcal{R}_t|} \sum_{(x_i, y_i) \in \mathcal{R}_t} E(\hat{y}(x_i, \theta_{1,t-1}, \dots, \theta_{L,t-1}), y_i)$ ;
   /* Backward propagation */
4   for  $l = 1$  to  $L$  do
5     Compute the gradient:  $g_{l,t} = \nabla_{\theta_{l,t-1}} \mathcal{L}$ ;
     /*  $\hat{m}_{l,t} = \frac{m_{l,t}}{1-\beta_1^t}$ , and  $m_{l,t} = \beta_1 m_{l,t-1} + (1-\beta_1)g_{l,t}$  is equivalent to
         $m_{l,t} = \hat{\beta}_{1,t} m_{l,t-1} + (1-\hat{\beta}_{1,t})g_{l,t}$ , where  $\hat{\beta}_{1,t} = \frac{\beta_1 - \beta_1^t}{1-\beta_1^t}$ . The same holds for
        second-moment estimation */
6     First-moment estimate:  $m_{l,t} = \hat{\beta}_{1,t} m_{l,t-1} + (1-\hat{\beta}_{1,t})g_{l,t}$ ,  $\hat{\beta}_{1,t} = \frac{\beta_1 - \beta_1^t}{1-\beta_1^t}$ ;
7     Elementwise second-moment estimate:  $v_{l,t} = \hat{\beta}_{2,t} v_{l,t-1} + (1-\hat{\beta}_{2,t})g_{l,t}^2$ ,
         $\hat{\beta}_{2,t} = \frac{\beta_2 - \beta_2^t}{1-\beta_2^t}$ ;
8     Global second-moment estimate:  $n_{l,t} = \hat{\beta}_{2,t} n_{l,t-1} + (1-\hat{\beta}_{2,t})\|g_{l,t}\|_2^2$ ,
         $\hat{\beta}_{2,t} = \frac{\beta_2 - \beta_2^t}{1-\beta_2^t}$ ;
     /*  $|g_{l,t}|$  denotes the dimension of  $g_{l,t}$  */
9     Switching term:  $s_{l,t} = (v_{l,t})^{\alpha_t} \left(\frac{n_{l,t}}{|g_{l,t}|}\right)^{1-\alpha_t}$ ;
10    Parameter update:  $\theta_{l,t} = \theta_{l,t-1} - \eta_t \left(\frac{m_{l,t}}{\epsilon + \sqrt{s_{l,t}}} + \lambda_l \theta_{l,t-1}\right)$ ,
         $\lambda_l = \begin{cases} \lambda, & \text{if } \theta_l \text{ is a matrix parameter,} \\ 0, & \text{if } \theta_l \text{ is not a matrix parameter;} \end{cases}$ 
    /* Save the best loss value and extremal point */
11    if  $\mathcal{L}_t < \mathcal{L}_0$  then
12       $\mathcal{L}_0 \leftarrow \mathcal{L}_t$ ;
13      for  $l = 1$  to  $L$  do
14         $\theta_l^* \leftarrow \theta_{l,t}$ ;
    // Store the extremal point

```

---

## 4. Experimental results and analysis

We conducted systematic benchmark comparisons between Ada2MS and several optimization algorithms to evaluate its behavior under a unified optimizer-comparison protocol. Experiments were performed on three benchmark datasets: CIFAR-100 [26] for image classification, PASCAL VOC [27] for object detection, and Semantic Boundaries [28] for semantic segmentation. SwinV2 [29] was selected as the standard baseline for image classification, YOLOv7-tiny [30] was selected as the benchmark model for object detection, and U-Net [31] was selected as the baseline architecture for semantic segmentation.

### 4.1. Reproduction details

This section presents the experimental details to support reproducibility. Table 1 lists the main software and hardware configurations used in the experiments.

Table 1: Main software and hardware configurations used in the experiments

|                     |                          |
|---------------------|--------------------------|
| Torch [32]          | 2.9.0                    |
| Torchvision [32]    | 0.24.0                   |
| Python              | 3.13.7                   |
| NVIDIA CUDA Toolkit | 13.0                     |
| RAM                 | 192 GB                   |
| CPU                 | Intel(R) Core i9-14900KF |
| GPU                 | NVIDIA RTX A5000         |

Learning-rate strategy configuration. Two learning-rate strategies were used for different models: Warmup-Steady-Decay-Steady (WSDS) and Warmup-Steady-Decay (WSD), as shown in Equations (29) and (30), respectively:

$$\eta_t = \begin{cases} \eta_{\text{init}} + \frac{\eta_{\text{peak}} - \eta_{\text{init}}}{t_1} t, & \text{if } 0 < t \leq t_1, \\ \eta_{\text{peak}}, & \text{if } t_1 < t \leq t_2, \\ \eta_{\text{peak}} + \frac{\eta_{\text{min}} - \eta_{\text{peak}}}{t_3 - t_2} (t - t_2), & \text{if } t_2 < t \leq t_3, \\ \eta_{\text{min}}, & \text{if } t > t_3 \end{cases} \quad (29)$$

$$\eta_t = \begin{cases} \eta_{\text{init}} + \frac{\eta_{\text{peak}} - \eta_{\text{init}}}{t_1} t, & \text{if } 0 < t \leq t_1, \\ \eta_{\text{peak}}, & \text{if } t_1 < t \leq t_2, \\ \eta_{\text{peak}} + \frac{\eta_{\text{min}} - \eta_{\text{peak}}}{t_3 - t_2} (t - t_2), & \text{if } t_2 < t \leq t_3 \end{cases} \quad (30)$$

Here,  $\eta_{\text{init}}$  denotes the initial learning rate,  $\eta_{\text{peak}}$  denotes the peak learning rate, and  $\eta_{\text{min}}$  denotes the minimum learning rate. Figure 1 visualizes the functional relationship between the WSDS learning rate and the number of iterations.

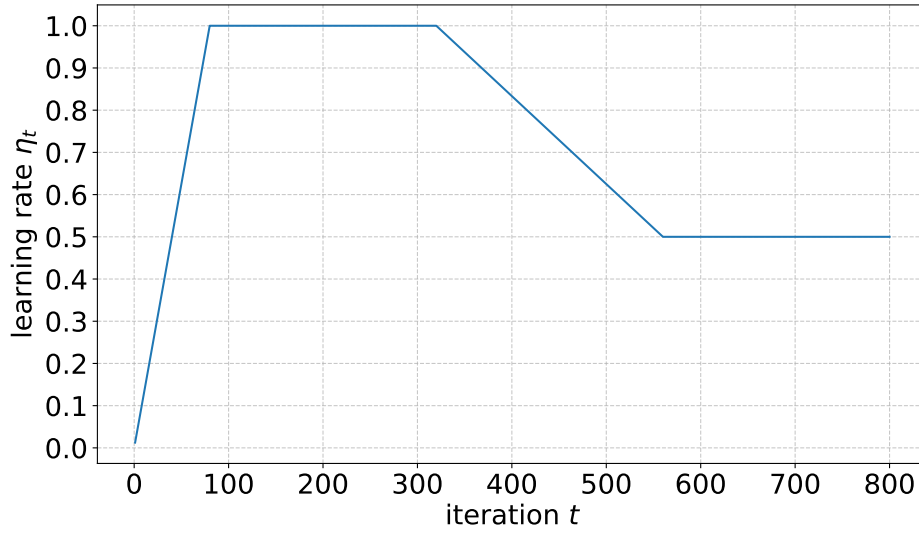


Figure 1: Function curve between the WSDS learning rate and the number of iterations

Note: The horizontal and vertical axes in the figure are set only to intuitively show the trend of the learning rate over the iteration process; the numerical values do not reflect the real data used in the experiments.

One difficulty in research on optimization algorithms is how to design experiments that compare different optimizers relatively fairly, because different optimizers have different hyperparameter configurations. Aligning learning rates and weight-decay rates through the RMS norm of the update is relatively fair, namely

$$\begin{cases} \eta_{\text{opt}_2} = \eta_{\text{opt}_1} \frac{\|\mathbf{u}_{\text{opt}_1}\|_2}{\|\mathbf{u}_{\text{opt}_2}\|_2}, \\ \lambda_{\text{opt}_2} = \lambda_{\text{opt}_1} \frac{\|\mathbf{u}_{\text{opt}_2}\|_2}{\|\mathbf{u}_{\text{opt}_1}\|_2} \end{cases} \quad (31)$$

Table 2 lists the peak learning rates of different optimization algorithms on different models. In addition, the initial warmup learning rate of each optimizer was uniformly

set to  $1 \times 10^{-7}$ , and the minimum learning rate was uniformly set to 0.01 times the peak learning rate. The weight-decay rates of AdamW, RAdam, and Ada2MS were uniformly set to 0.01, whereas the weight-decay rates of momentum SGD, AdaI, Lion, and SophiaG were computed according to Equation (31).

Table 2: Peak learning rates of different optimization algorithms on different models

|                  | SwinV2-S [29]          | YOLOv7-tiny [30]      | U-Net [31]            |
|------------------|------------------------|-----------------------|-----------------------|
| Momentum SGD [8] | $2.67 \times 10^{-2}$  | 0.349                 | 0.116                 |
| AdamW [5]        | $5.34 \times 10^{-4}$  | $3.49 \times 10^{-3}$ | $1.16 \times 10^{-3}$ |
| RAdam [21]       | $5.34 \times 10^{-4}$  | $3.49 \times 10^{-3}$ | $1.16 \times 10^{-3}$ |
| AdaI [7]         | 1                      | 1                     | 1                     |
| Lion [24]        | $1.068 \times 10^{-4}$ | $6.98 \times 10^{-4}$ | $2.32 \times 10^{-4}$ |
| SophiaG [20]     | $1.068 \times 10^{-4}$ | $6.98 \times 10^{-4}$ | $2.32 \times 10^{-4}$ |
| Ada2MS (Ours)    | $5.34 \times 10^{-4}$  | $3.49 \times 10^{-3}$ | $1.16 \times 10^{-3}$ |

#### 4.2. Image classification results and analysis

Image classification benchmark dataset. CIFAR-100 [26] was created by the Department of Computer Science at the University of Toronto in 2009 as an expanded version of the CIFAR series. It contains 60,000 RGB color images of size  $32 \times 32$ , forming a fine-grained classification framework with 100 classes grouped into 20 superclasses, each containing 5 subclasses. The dataset is strictly split into 50,000 training images and 10,000 test images. Random color-jitter transformation and random horizontal flipping were used for data augmentation during training.

Image classification benchmark model. SwinV2 [29] is an enhanced visual Transformer developed by Microsoft, with a series of innovations for training large models. To adapt it to the CIFAR resolution, the patch size of SwinV2 was set to 1 and the window size was set to 4. The model was reproduced from the official torchvision implementation. Weight initialization followed the official default strategy, and no pre-trained parameters were loaded.

Image classification evaluation metrics. Top-1 error rate and Top-5 error rate are standard metrics for evaluating image classification models. The test set contains  $N$  images. Let the ground-truth label of the  $i$ -th image be  $y_i$ , and let the class-probability vector predicted by the model for that image be  $p_i = (p_{i,1}, p_{i,2}, \dots, p_{i,C})$ , where  $C$  is the total number of classes. The two error rates are defined as follows:

$$E_{\text{Top-1}} = \frac{1}{N} \sum_{i=1}^N \chi_A \left( y_i \neq \arg \max_j p_{i,j} \right), \quad (32)$$

$$E_{\text{Top-5}} = \frac{1}{N} \sum_{i=1}^N \chi_A \left( y_i \notin \text{top-5} \left( p_{i,j=1}^C \right) \right) \quad (33)$$

Here,  $\chi_A(\cdot)$  denotes the indicator function. This definition follows the convention of [33] and has become an evaluation standard for subsequent research.

Optimal performance comparison. Table 3 reports the results of training SwinV2-S on CIFAR-100 with different optimization algorithms. The number of epochs was set to 160, the batch size was set to 64, the WSDS learning-rate strategy was used, and the peak learning rates followed Table 2. Ada2MS achieved the second-best Top-1 error rate of 29.17%, only 0.5% higher than the lowest Top-1 error rate of 28.67%.

Table 3: Image classification results of different optimization algorithms on CIFAR-100

|                  | Top-1 error rate (%)↓ | Top-5 error rate (%)↓ |
|------------------|-----------------------|-----------------------|
| Momentum SGD [8] | 33.22                 | 10.73                 |
| AdamW [5]        | 29.38                 | 8.39                  |
| RAdam [21]       | 29.38                 | <u>8.37</u>           |
| AdaI [7]         | 31.34                 | 10.03                 |
| Lion [24]        | <b>28.67</b>          | <b>8.17</b>           |
| SophiaG [20]     | 29.84                 | 8.87                  |
| Ada2MS (Ours)    | <u>29.17</u>          | 8.42                  |

Note: Bold denotes the best result, and underline denotes the second-best result.

#### 4.3. Object detection results and analysis

Object detection benchmark dataset. Following the standard protocol in PASCAL VOC [27] object detection, the training data were formed by combining the trainval

subsets of VOC2007 and VOC2012, with 16,551 images covering 20 object classes. Model evaluation was conducted on the publicly annotated VOC2007 test set, which contains 4,952 images. Random padding and scaling, color-jitter transformation, and horizontal flipping were used as data augmentation during training.

Object detection benchmark model. YOLOv7-tiny is a lightweight and fast variant of the YOLOv7 [30] series. Based on the compact ELAN-Tiny backbone, it achieves millisecond-level inference while maintaining competitive detection accuracy, balancing accuracy and efficiency. The input image resolution was set to  $320 \times 320$ . Model weight initialization followed the RegNet [34] initialization strategy.

Object detection evaluation metric. Mean Average Precision (mAP) is the most widely used evaluation metric in object detection. Its computation is as follows. For each class  $j$ , all predicted boxes are sorted in descending order by confidence and compared one by one with the ground-truth boxes. If  $\text{IoU} \geq T$ , for example  $T = 0.5$  or  $0.75$ , and the class is correct, the prediction is counted as a True Positive (TP); otherwise, it is counted as a False Positive (FP). Precision and recall are then accumulated as

$$P_j(k) = \frac{\sum_{i=1}^k \text{TP}_i}{k}, \quad (34)$$

$$R_j(k) = \frac{\sum_{i=1}^k \text{TP}_i}{N_k} \quad (35)$$

Here,  $N_j$  denotes the total number of ground-truth boxes in class  $j$ , and  $k$  denotes the top  $k$  predicted boxes. The curve  $P_j(R_j)$  is plotted, and the area under this curve gives the average precision (AP) of the class:

$$\text{AP}_j = \int_0^1 P_j(R_j) dR_j \quad (36)$$

Averaging over all classes gives  $\text{mAP} = \frac{1}{C} \sum_{j=1}^C \text{AP}_j$ .

Optimal performance comparison. Table 4 reports the results of training YOLOv7-tiny on VOC2007 and VOC2012 with different optimization algorithms. The number of epochs was set to 80, the batch size was set to 32, the WSD learning-rate strategy was used, and the peak learning rates followed Table 2. Ada2MS achieved the highest mAP@0.5 of 59.40% and the highest mAP@0.75 of 35.46%.

Table 4: Object detection results of different optimization algorithms on VOC2007 test

|                  | mAP@0.5 (%) $\uparrow$ | mAP@0.75 (%) $\uparrow$ |
|------------------|------------------------|-------------------------|
| Momentum SGD [8] | 57.19                  | 31.02                   |
| AdamW [5]        | 59.12                  | 34.91                   |
| RAdam [21]       | 58.89                  | 34.66                   |
| AdaI [7]         | 56.91                  | 31.31                   |
| Lion [24]        | 56.97                  | 32.14                   |
| SophiaG [20]     | 54.61                  | 29.07                   |
| Ada2MS (Ours)    | <b>59.40</b>           | <b>35.46</b>            |

Note: mAP@0.5 denotes  $\text{IoU} \geq 0.5$ , and mAP@0.75 denotes  $\text{IoU} \geq 0.75$ .

#### 4.4. Semantic segmentation results and analysis

Semantic segmentation benchmark dataset. The Semantic Boundaries [28] dataset contains 11,355 images from PASCAL VOC 2011. It is split into 8,498 training images and 2,857 validation images, and provides pixel-level semantic segmentation annotations and boundary information for 20 object classes, covering semantic segmentation and boundary detection tasks. Random scaling, horizontal flipping, and random cropping were used as data augmentation during training.

Semantic segmentation benchmark model. U-Net [31] was originally proposed for biomedical image segmentation. Its symmetric encoder-decoder architecture uses skip connections to fuse high-resolution details and low-resolution semantic information, achieving accurate pixel-level prediction with limited labeled data. This design balances localization accuracy and contextual modeling capability, making U-Net a widely used baseline model in semantic segmentation. The number of basic units in U-Net was set to 64, and the input image resolution was set to  $320 \times 320$ . Model weight initialization followed the RegNet [34] initialization strategy. The loss function used a weighted average of cross-entropy loss [11] and Dice loss [35].

Semantic segmentation evaluation metric. Mean Intersection over Union (mIoU) is a standard metric in semantic segmentation. It is computed by calculating the intersection-over-union between the prediction and the ground truth for each class and then taking

the average:

$$\text{mIoU} = \frac{1}{C} \sum_{i=1}^C \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i + \text{FN}_i} \quad (37)$$

Here,  $C$  denotes the total number of classes, and  $\text{TP}_i$ ,  $\text{FP}_i$ , and  $\text{FN}_i$  denote the number of true-positive, false-positive, and false-negative pixels for class  $i$ , respectively. This metric measures both localization precision and classification accuracy; higher values indicate better segmentation performance.

Optimal performance comparison. Table 5 reports the results of training U-Net on Semantic Boundaries with different optimization algorithms. The number of epochs was set to 80, the batch size was set to 32, the WSD learning-rate strategy was used, and the peak learning rates followed Table 2. Ada2MS achieved the third-best mIoU of 52.52%.

Table 5: Semantic segmentation results of different optimization algorithms on Semantic Boundaries

|                  | mIoU (%) $\uparrow$ |
|------------------|---------------------|
| Momentum SGD [8] | 52.40               |
| AdamW [5]        | 52.28               |
| RAdam [21]       | <u>52.83</u>        |
| AdaI [7]         | <b>53.50</b>        |
| Lion [24]        | 50.27               |
| SophiaG [20]     | 47.92               |
| Ada2MS (Ours)    | <u>52.52</u>        |

Note: Wavy underline denotes the third-best result.

## 5. Conclusion and future work

Ada2MS realizes a smooth transition from AdamW-like behavior to momentum-SGD-like behavior through continuous interpolation between elementwise second-moment normalization and global second-moment normalization. The goal of this mechanism is to gradually introduce more SGD-oriented update characteristics while maintaining update-scale stability as much as possible. Under the current evaluation protocol, Ada2MS obtained competitive results on the CIFAR-100 classification task and

achieved the best results among the compared optimizers on the VOC detection task. These results indicate that the mechanism has potential. However, more hybrid baselines, more complete ablation studies of the switching schedule, and validation on non-visual tasks remain necessary in future work.

## Acknowledgments

This work was partly supported by the Jiangxi Provincial Key Laboratory of Virtual Reality (Grant No. 2024SSY03151).

## Data availability

The datasets during the current study are open source and available in <http://www.cs.toronto.edu/~kriz/cifar.html>, <https://www.kaggle.com/datasets/zaraks/pascal-voc-2007>, <https://www.kaggle.com/datasets/huanghanchina/pascal-voc-2012>, [https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/semantic\\_contours/benchmark.tgz](https://www2.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/semantic_contours/benchmark.tgz) repositories, respectively.

## References

- [1] J. Jumper, R. Evans, A. Pritzel, et al., Highly accurate protein structure prediction with alphafold, *Nature* 596 (7873) (2021) 583–589. doi:10.1038/s41586-021-03819-2.
- [2] A. Merchant, S. Batzner, S. S. Schoenholz, et al., Scaling deep learning for materials discovery, *Nature* 624 (7990) (2023) 80–85. doi:10.1038/s41586-023-06735-9.
- [3] S. K. Kim, R. Shousha, S. M. Yang, et al., Highest fusion performance without harmful edge energy bursts in tokamak, *Nature Communications* 15 (1) (2024) 3990–4001. doi:10.1038/s41467-024-48415-w.

- [4] H. Touvron, L. Martin, K. Stone, et al., Llama 2: Open foundation and fine-tuned chat models (2023).  
URL <https://arxiv.org/abs/2307.09288>
- [5] I. Loshchilov, F. Hutter, Fixing weight decay regularization in adam (2017).  
URL <http://arxiv.org/abs/1711.05101>
- [6] N. S. Keskar, R. R. Socher, Improving generalization performance by switching from adam to sgd (2017).  
URL <http://arxiv.org/abs/1712.07628>
- [7] Z. Xie, X. Wang, H. Zhang, et al., Adaptive inertia: Disentangling the effects of adaptive learning rate and momentum, in: International Conference on Machine Learning, Vol. 162, 2022, pp. 24430–24459.
- [8] I. Sutskever, J. Martens, G. Dahl, G. Hinton, On the importance of initialization and momentum in deep learning, in: International Conference on Machine Learning, 2013, p. 1139–1147.
- [9] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., An image is worth 16x16 words: Transformers for image recognition at scale, in: International Conference on Learning Representations, 2021, pp. 1–21.
- [10] Z. Liu, H. Mao, C. Y. Wu, et al., A convnet for the 2020s, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 11966–11976. doi:10.1109/CVPR52688.2022.01167.
- [11] D. E. Rumelhart, G. E. Hinton, R. J. Williams, Learning representations by back-propagating errors, *Nature* 323 (6088) (1986) 533–536. doi:10.1038/323533a0.
- [12] I. Gitman, H. Lang, P. Zhang, L. Xiao, Understanding the role of momentum in stochastic gradient methods, in: International Conference on Neural Information Processing Systems, Vol. 32, 2019, pp. 1–11.
- [13] A. Ramezani-Kebrya, K. Antonakopoulos, V. Cevher, et al., On the generalization of stochastic gradient descent with momentum, *Journal of Machine Learning Research* 25 (22) (2024) 1–56.

- [14] S. Zhao, C. Shi, Y. Xie, W. Li, Stochastic normalized gradient descent with momentum for large-batch training, *SCIENCE CHINA Information Sciences* 67 (11) (2024) 1–15. doi:10.1007/s11432-022-3892-8.
- [15] J. Duchi, E. Hazan, Y. Singer, Adaptive subgradient methods for online learning and stochastic optimization, *The Journal of Machine Learning Research* 12 (2011) 2121–2159.
- [16] T. Tieleman, G. Hinton, Rmsprop: Divide the gradient by a running average of its recent magnitude (2012).  
URL <https://cir.nii.ac.jp/crid/1370017282431050757>
- [17] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: *International Conference on Learning Representations*, 2015, pp. 1–15.
- [18] S. J. Reddi, S. Kale, S. Kumar, On the convergence of adam and beyond, in: *International Conference on Learning Representations*, 2018, pp. 1–23.
- [19] N. Shazeer, M. Stern, Adafactor: Adaptive learning rates with sublinear memory cost, in: *International Conference on Machine Learning*, Vol. 80, 2018, pp. 4596–4604.
- [20] H. Liu, Z. Li, D. L. W. Hall, et al., Sophia: A scalable stochastic second-order optimizer for language model pre-training, in: *International Conference on Learning Representations*, 2024, pp. 1–30.
- [21] L. Liu, H. Jiang, P. He, et al., On the variance of the adaptive learning rate and beyond, in: *International Conference on Learning Representations*, 2020, pp. 1–13.
- [22] W. Li, Z. Zhang, X. Wang, P. Luo, Adax: Adaptive gradient descent with exponential long term memory (2020).  
URL <https://openreview.net/forum?id=r1l-5pEtDr>
- [23] M. Zhu, Q. Xiao, W. Min, Adamnx: An adam improvement algorithm based on a novel exponential decay mechanism for the second-order moment estimate

(2025).

URL <https://arxiv.org/abs/2511.13465>

- [24] X. Chen, C. Liang, D. Huang, et al., Symbolic discovery of optimization algorithms, in: International Conference on Neural Information Processing Systems, 2023, pp. 1–30.
- [25] K. Jordan, Y. Jin, V. Boza, et al., Muon: An optimizer for hidden layers in neural networks (2024).  
URL <https://kellerjordan.github.io/posts/muon>
- [26] A. Krizhevsky, G. Hinton, Learning multiple layers of features from tiny images (2009).  
URL <https://citeseerx.ist.psu.edu>
- [27] M. Everingham, J. Winn, The pascal visual object classes challenge 2012 (voc2012) development kit, Pattern Analysis, Statistical Modelling and Computational Learning, Tech. Rep 8.
- [28] B. Hariharan, P. Arbeláez, L. Bourdev, et al., Semantic contours from inverse detectors, in: IEEE/CVF International Conference on Computer Vision, 2011, pp. 991–998. doi:10.1109/ICCV.2011.6126343.
- [29] Z. Liu, H. Hu, Y. Lin, et al., Swin transformer v2: Scaling up capacity and resolution, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2022, pp. 12009–12019.
- [30] C. Wang, A. Bochkovskiy, H. Liao, Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2023, pp. 7464–7475. doi:10.1109/CVPR52729.2023.00721.
- [31] O. Ronneberger, P. Fischer, T. Brox, U-net: Convolutional networks for biomedical image segmentation, in: Medical Image Computing and Computer-Assisted Intervention, 2015, pp. 234–241.

- [32] A. Paszke, S. Gross, F. Massa, et al., Pytorch: An imperative style, high-performance deep learning library, in: International Conference on Neural Information Processing Systems, Vol. 32, 2019, pp. 1–12.
- [33] A. Krizhevsky, I. Sutskever, G. E. Hinton, Imagenet classification with deep convolutional neural networks, in: International Conference on Neural Information Processing Systems, Vol. 25, 2012, pp. 1–9.
- [34] I. Radosavovic, R. P. Kosaraju, R. Girshick, et al., Designing network design spaces, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020, pp. 10428–10436.
- [35] F. Milletari, N. Navab, S. A. Ahmadi, V-net: Fully convolutional neural networks for volumetric medical image segmentation, in: IEEE/CVF International Conference on 3D Vision, 2016, pp. 565–571. doi:10.1109/3DV.2016.79.